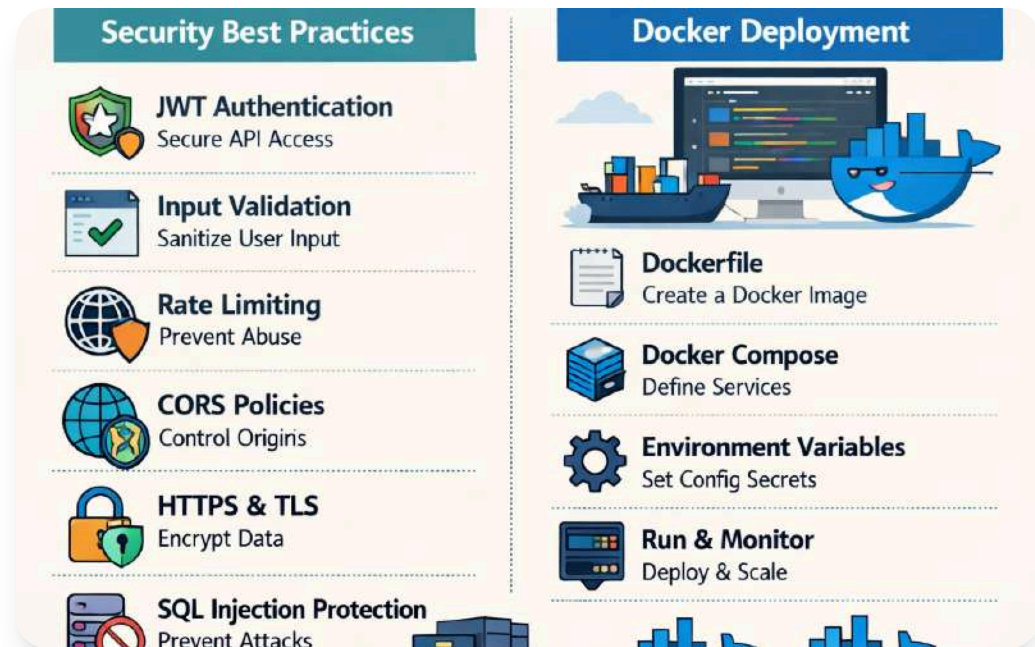




Golang Gin Backend Pertemuan 20

Materi mencakup security API, rate limiter, CORS, Dockerfile, dan Docker Compose, plus 2 latihan praktik intermediate.



Juara Coding

June 2026



Tujuan Pembelajaran

Memahami risiko backend Gin, menerapkan rate limiter, mengatur CORS, membuat Dockerfile Go, dan memakai Docker Compose.



Security Backend dengan Gin

Keamanan backend mencakup header sensitif, kontrol origin, validasi input, error aman, dan deployment konsisten lewat middleware Gin.

Ancaman Umum pada API



Brute Force dan Spam Request

Login berulang dan request berlebihan dapat mengganggu layanan jika tidak dibatasi.



Error dan Konfigurasi Tidak Aman

Pesan error yang bocor serta container yang salah konfigurasi meningkatkan risiko paparan sistem.



Validasi dan CORS Lemah

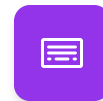
Input tidak tervalidasi dan CORS terlalu longgar membuka celah penyalahgunaan API.



Praktik Security Dasar di Gin



Terapkan validasi input, lindungi konfigurasi sensitif, dan batasi akses route untuk memperkuat keamanan aplikasi Gin.



Validasi Input

Gunakan binding dan validator secara konsisten untuk memeriksa semua input dari client.



Lindungi Konfigurasi

Simpan data sensitif di environment variable dan jangan tampilkan stack trace ke client.



Batasi Akses Request

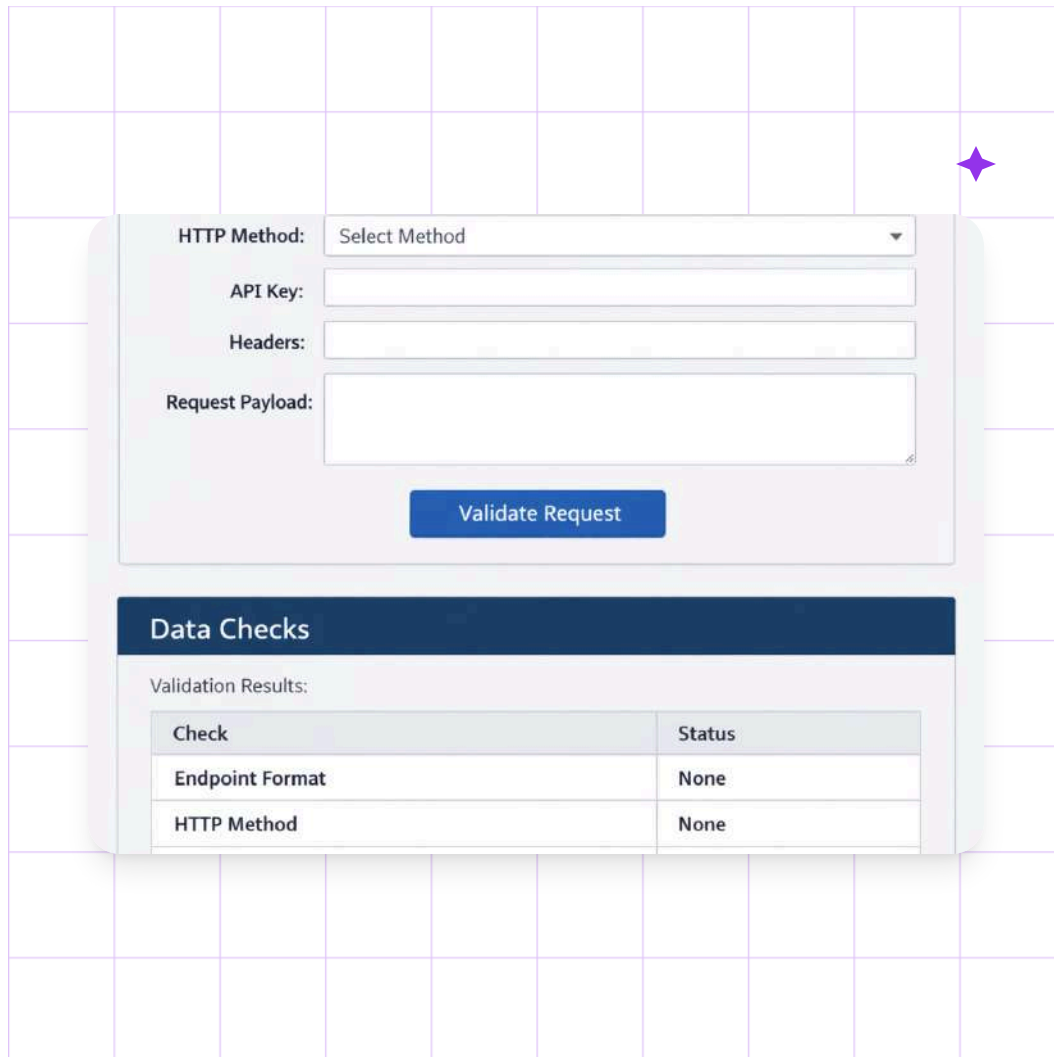
Buka hanya method dan route yang diperlukan, lalu tambahkan middleware keamanan.

```
func SecurityHeadersMiddleware(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r  
                                http.Request) {  
  
        w.Header().Set("Strict-Transport-Security", "max-age=31536000;  
                                includeSubDomains",  
        w.Header().Set("Content-Security-Policy", "default-src \"self\";  
                                script-src \"self\"]  
        w.Header().Set("X-Content-Type-Options", "nosniff",  
        w.Header().Set("X-Frame-Options", "DENY",  
        w.Header().Set("X-XSS-Protection", "1; mode=block",  
        w.Header().Set("Referrer-Policy", "no-referrer",  
  
        next.ServeHTTP(w, r)
```

Contoh Middleware Security Header

Middleware ini menambahkan header dasar untuk membantu mengurangi risiko serangan berbasis browser pada respons aplikasi.

Validasi Input Request



The screenshot shows a web interface for validating API requests. It features a form with the following fields:

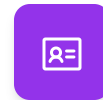
- HTTP Method:** A dropdown menu currently showing "Select Method".
- API Key:** A text input field.
- Headers:** A text input field.
- Request Payload:** A larger text input area.

Below the form is a blue button labeled "Validate Request".

Underneath the button is a section titled "Data Checks" with a dark blue header. Below this header, it says "Validation Results:" followed by a table:

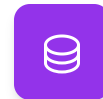
Check	Status
Endpoint Format	None
HTTP Method	None

Validasi input mencegah data tidak valid masuk ke sistem sejak awal proses request.



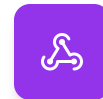
Field wajib

Gunakan struct binding dan tag validasi untuk memastikan input penting selalu terisi.



Cek format data

Validasi email, panjang karakter, dan nilai numerik sebelum request diproses lebih lanjut.



Penting untuk endpoint

Sangat berguna pada registrasi, login, dan pembuatan resource agar data tetap konsisten.

```
type User struct {
    Username string "json:\"username\" binding:\"required\""
    Email    string "json:\"email\" binding:\"required,email\""
}

func main() {
    r = gin.Default()

    r.POST("/register", func(c *gin.Context) {
        var newUser User

        if err := c.ShouldBindJSON(&newUser); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
            return
        }
    })
}
```

Contoh Validasi di Gin

Gin menolak request yang tidak valid sejak awal, sehingga data salah tidak masuk ke proses bisnis aplikasi.



Penanganan Error yang Aman

Respons error harus informatif bagi client tanpa membocorkan detail internal. Simpan detail asli di log server untuk kebutuhan debugging yang aman.

Konsep Rate Limiter



01 Membatasi request

Rate limiter membatasi jumlah request dari client dalam periode waktu tertentu agar akses tetap terkendali.

02 Mencegah penyalahgunaan

Digunakan untuk mengurangi risiko penyalahgunaan API, serangan brute force, dan aktivitas berlebihan.

03 Penerapan umum

Sering diterapkan pada endpoint login, OTP, registrasi, dan endpoint publik dengan trafik tinggi.

Strategi Rate Limiting



01 Berdasarkan identitas request

Pembatasan umum dapat diterapkan per IP address, API key, user ID, atau route tertentu sesuai kebutuhan layanan.

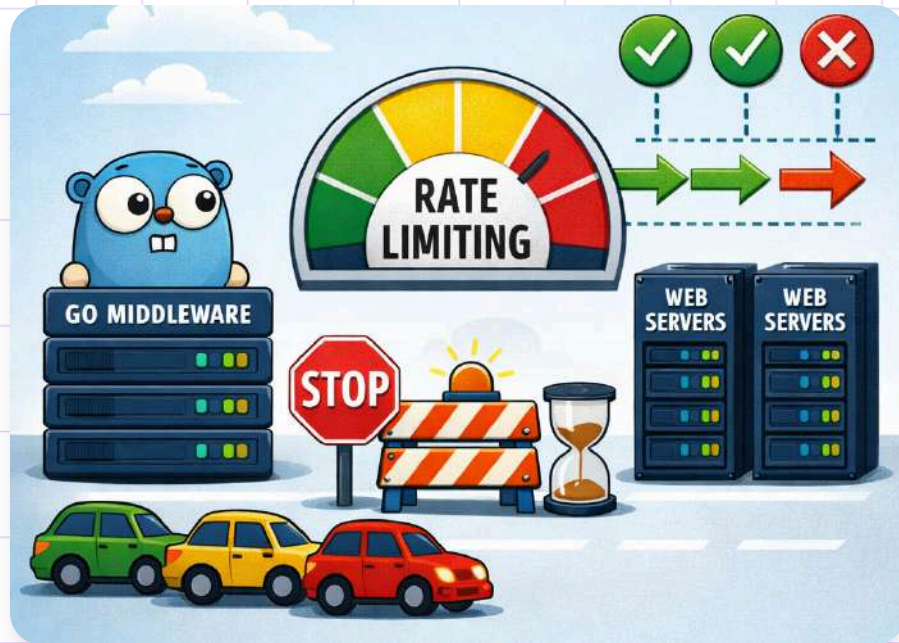
02 Pendekatan sederhana untuk skala kecil

Metode dasar cocok untuk aplikasi kecil dengan arsitektur sederhana dan kebutuhan kontrol trafik yang belum kompleks.

03 Redis untuk sistem terdistribusi

Aplikasi terdistribusi biasanya menyimpan limit di Redis agar aturan tetap konsisten antar instance container atau server.

Contoh Rate Limiter Sederhana



Implementasi ini membatasi request secara sederhana di level aplikasi menggunakan middleware Go.



Limiter Global

`rate.NewLimiter(1, 3)` mengatur laju 1 request per detik dengan burst hingga 3 request.



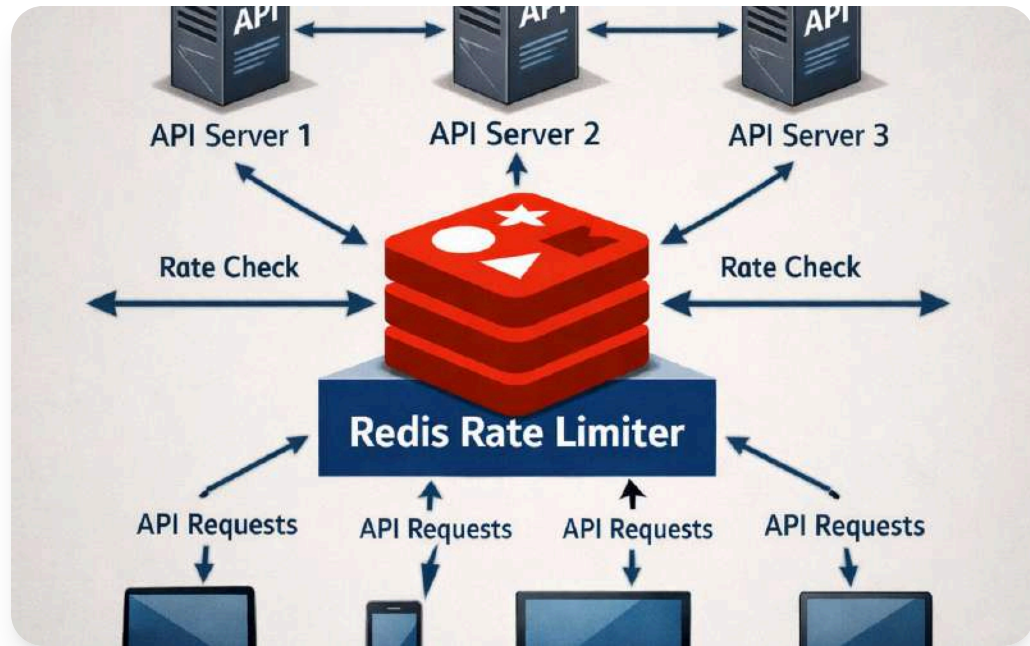
Cegah Akses Berlebih

Jika `Allow()` gagal, middleware mengembalikan status 429 dan menghentikan proses request.



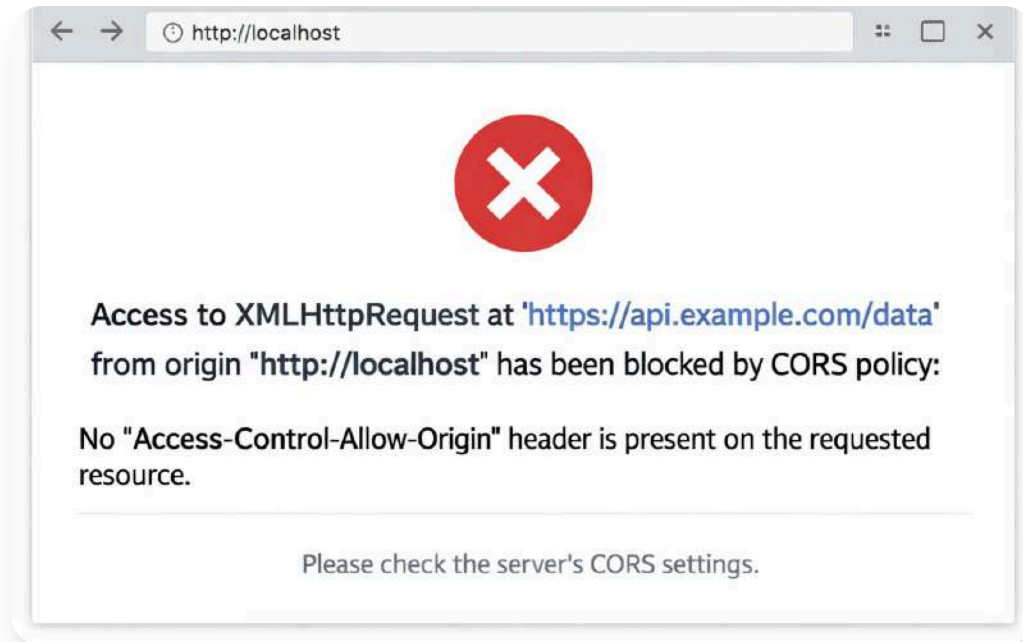
Level Aplikasi

Pendekatan ini mudah diterapkan, tetapi cocok untuk kebutuhan sederhana dan belum per pengguna.



Keterbatasan Rate Limiter

Rate limiter in-memory mudah diterapkan, tetapi data hilang saat restart, kurang akurat di multi-instance, dan tidak ideal untuk skala besar.



Konsep CORS

CORS mengatur akses browser ke resource beda origin. Penting saat frontend dan backend memakai domain atau port berbeda.

Risiko Konfigurasi CORS



CORS yang terlalu longgar atau terlalu ketat dapat mengganggu keamanan API dan akses frontend resmi.



Terlalu Longgar

Mengizinkan semua origin membuka peluang akses dari sumber yang tidak seharusnya.



Terlalu Ketat

Pembatasan berlebih dapat membuat frontend resmi gagal mengakses API.



Aturan Spesifik

Origin, method, header, dan credentials harus diatur sesuai kebutuhan aplikasi.



Contoh Middleware CORS Gin

Konfigurasi *CORS* ini mengizinkan frontend lokal mengakses backend *Gin* dengan origin, method, header, dan kredensial yang sesuai.



Credential Mode: "none"
No credentials sent with requests

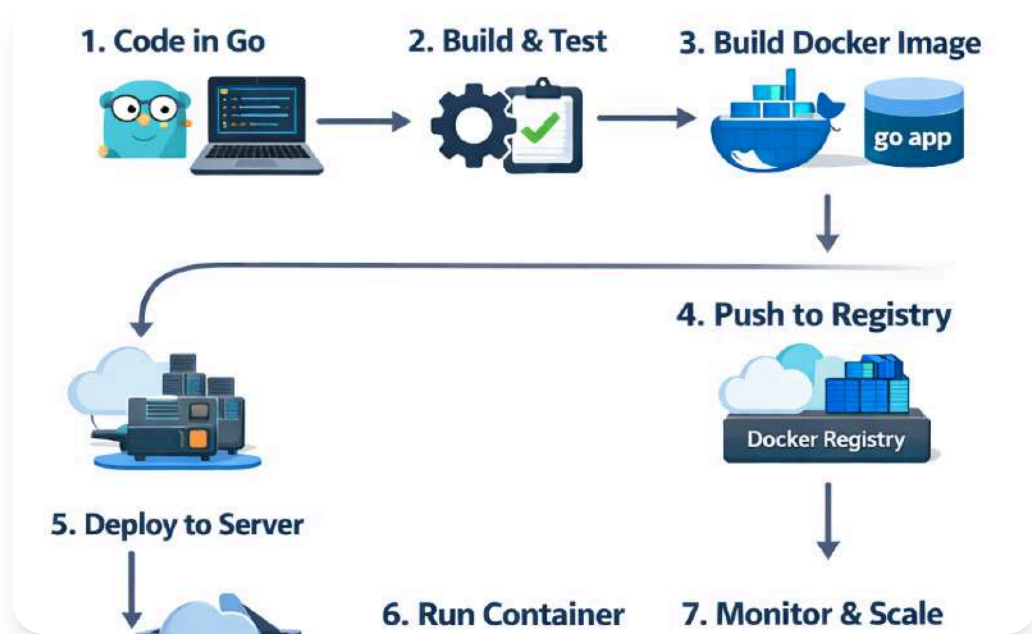
- No Cookies, No Session Data
- No Authorization Headers
- Anonymous Access Only

! Least Secure Setting
Cross-site requests cannot access user credentials.

- Used for public APIs and open resources

Kapan Menggunakan AllowCredentials

Digunakan saat frontend mengirim cookie, otorisasi browser, atau session lintas origin. Origin harus ditulis eksplisit agar akses tetap aman.



Docker untuk Backend Go

Docker membungkus aplikasi dan dependency ke image konsisten agar backend Gin berjalan sama di development, testing, staging, dan production.

Struktur Dasar Dockerfile Go

01 Base image dan workspace

Dockerfile dimulai dari base image, lalu menetapkan working directory sebagai lokasi proses build aplikasi.

03 Build binary dan jalankan aplikasi

Aplikasi Go dikompilasi menjadi binary di dalam container, lalu dijalankan melalui command saat container aktif.

02 Salin file dan unduh dependensi

File proyek disalin ke container, kemudian dependensi diunduh agar proses build dapat berjalan konsisten.

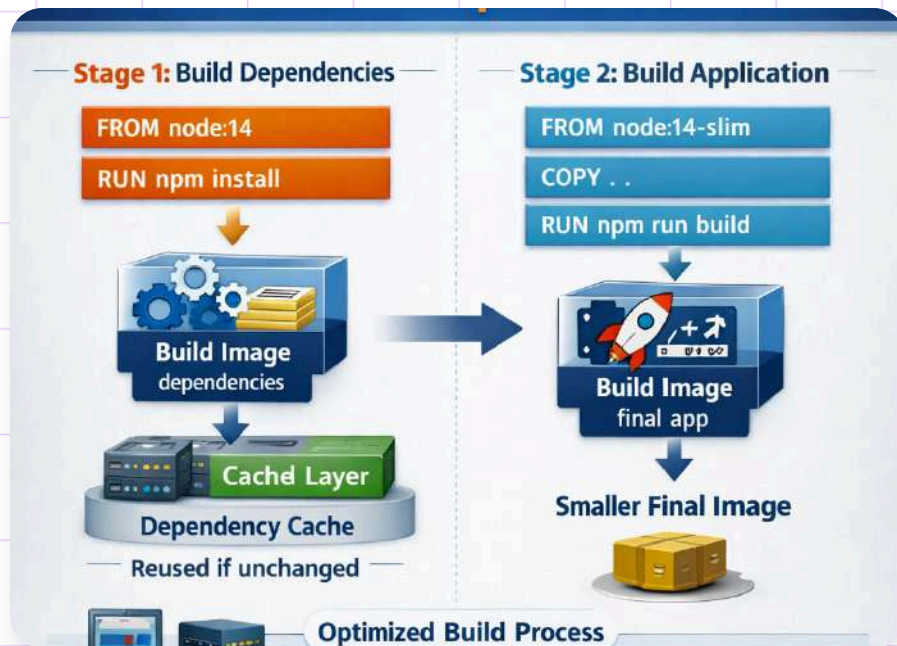
```
2  
3 WORKDIR /app  
4  
5 COPY package.json package-lock.json ./  
6 RUN npm install  
7 COPY . .  
8 RUN npm run build  
9  
10 EXPOSE 3000  
11 CMD ["node", "server.js"]
```

```
1 FROM golang:1.17-alpine
2 WORKDIR /app
3 COPY . .
4 RUN go build -o myapp .
5 CMD ["./myapp"]
```

Contoh Dockerfile Sederhana

Dockerfile ini membangun aplikasi Gin sederhana, menyalin kode ke /app, lalu menjalankannya pada port **8080** di dalam container.

Optimasi Dockerfile



Pisahkan copy go.mod dan go.sum untuk cache yang lebih efektif, lalu gunakan multi-stage build agar image runtime lebih kecil dan aman.



Cache Dependency

Copy go.mod dan go.sum lebih awal agar layer dependency dapat digunakan ulang saat build.



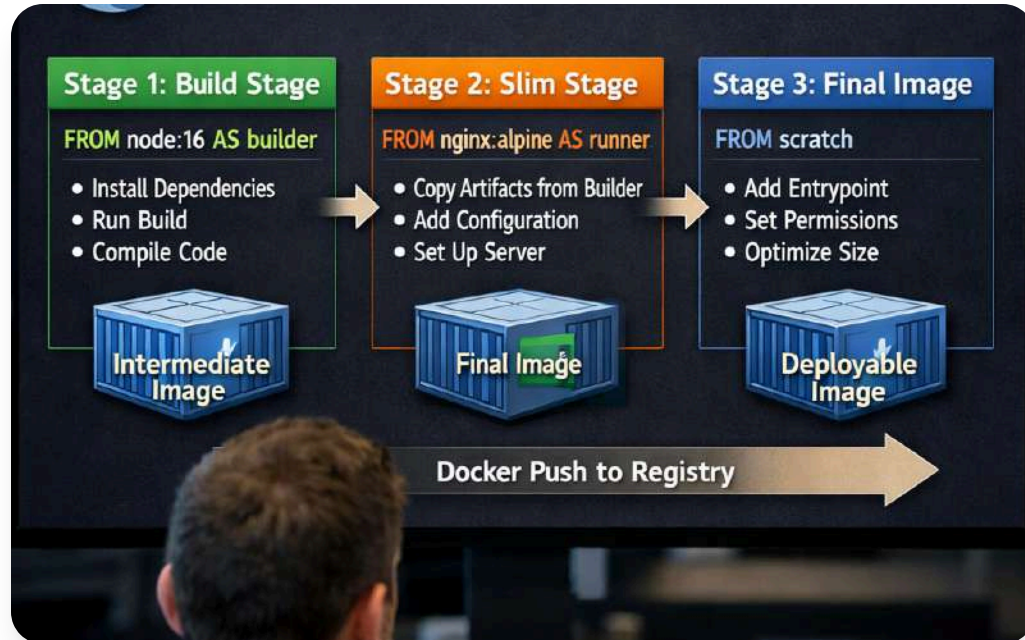
Multi-stage Build

Pisahkan tahap build dan runtime untuk menghasilkan image yang lebih ringan dan rapi.



Image Lebih Aman

Image runtime yang kecil mempercepat distribusi dan mengurangi permukaan serangan.



Contoh Multi-Stage Dockerfile

Pola ini memisahkan tahap build dan runtime sehingga menghasilkan image deployment yang lebih ringan dan efisien.

Docker Compose untuk Multi-Service

```
build: ./app
ports:
  - '5000:5000'
environment: > postgres://postgres:password@db:5432/
  codatabase
  - ENVIROMENT: production
db:
  image: postgres:13
  volumes:
    - db_data:/var/lib/postgresql/data
  environment:
    - POSTGRES_USER: postgres
    - POSTGRES_PASSWORD; password
    - POSTGRES_DB; mydatabase
  restart: always
```

01 Menjalankan banyak service

Docker Compose memungkinkan backend Gin dan database dijalankan bersama dalam satu stack aplikasi.

02 Satu file konfigurasi

Developer dapat mendefinisikan service, port, environment variable, volume, dan jaringan internal dalam satu file.

03 Proses deployment lebih sederhana

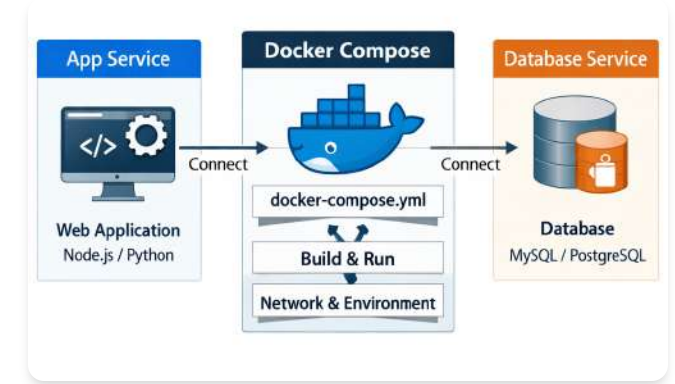
Konfigurasi terpusat membantu menjalankan aplikasi multi-service dengan lebih mudah, konsisten, dan efisien.

Contoh Docker Compose

Komponen	Konfigurasi
app	build: .; ports: 8080:8080; depends_on: db
db	image: postgres:16; POSTGRES_USER: postgres
db	POSTGRES_PASSWORD: postgres; POSTGRES_DB: app_db
db	ports: 5432:5432

Compose ini menghubungkan backend aplikasi dengan database PostgreSQL melalui dua service utama: *app* dan *db*.

Alur Kerja Docker Compose



01 Build image jika perlu

Saat *docker compose up* dijalankan, Compose akan membangun image aplikasi secara otomatis bila belum tersedia.

03 Akses database via nama service

Backend dapat terhubung ke database memakai nama service seperti *db*, bukan menggunakan *localhost*.

02 Buat network dan jalankan service

Compose membuat network internal, lalu menjalankan container *app* dan *db* dalam satu lingkungan terhubung.

Integrasi Materi dalam Project



Security dan Validasi

Security headers melindungi response, sementara validasi menjaga kualitas input pada aplikasi.



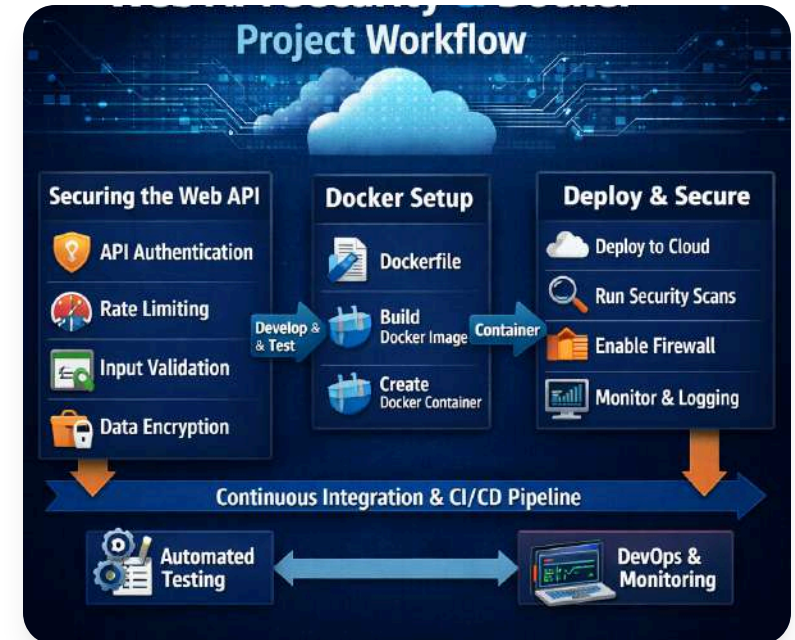
Konsistensi Deployment

Docker dan Compose memudahkan aplikasi dijalankan konsisten di environment pengembangan dan deployment.

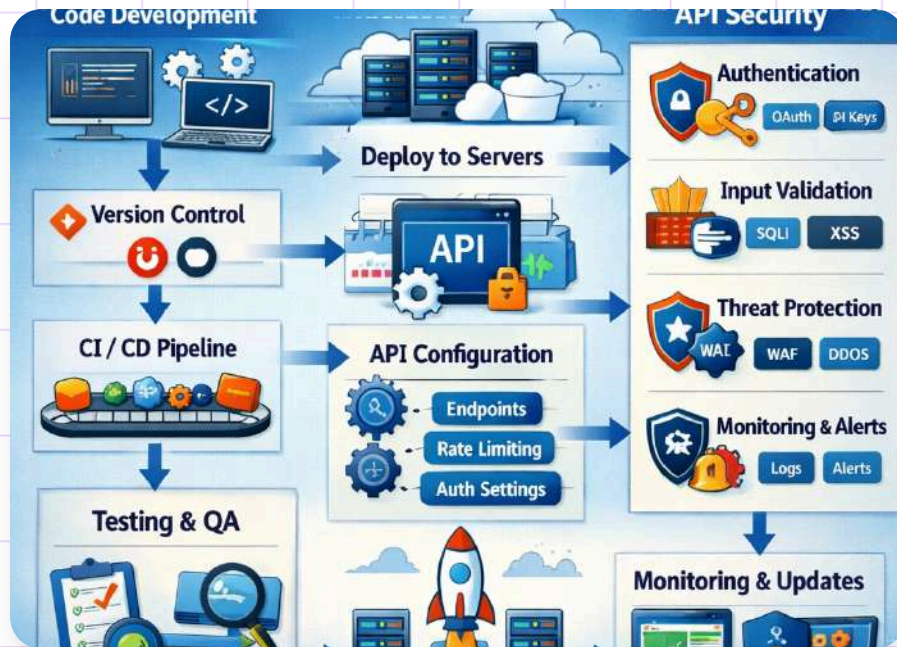


Proteksi Akses API

Rate limiter menjaga endpoint sensitif, dan CORS memungkinkan frontend resmi mengakses API.



Best Practice Implementasi

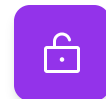


Pisahkan middleware, amankan konfigurasi, batasi CORS, dan siapkan deployment yang terdokumentasi.



Struktur Middleware

Pisahkan middleware sesuai fungsi agar alur aplikasi lebih jelas dan mudah dirawat.



Keamanan Konfigurasi

Simpan konfigurasi di environment variable dan batasi CORS untuk origin yang diperlukan.



Deployment Andal

Gunakan rate limiter, Dockerfile efisien, serta log dan health check yang terdokumentasi.

Latihan Praktik 1



01 Buat endpoint login

Implementasikan API login dengan Gin melalui endpoint POST /login untuk menerima email dan password pengguna.

03 Terapkan rate limiter

Batasi request pada endpoint login. Jika melewati batas, kembalikan status 429 dan uji dengan request berulang.

02 Tambahkan pengamanan

Lakukan validasi email dan password, lalu pasang middleware security headers agar respons API lebih aman.

Latihan Praktik 2

01 Containerisasi aplikasi Gin

Buat *Dockerfile multi-stage* untuk build dan menjalankan aplikasi Gin secara efisien di container.

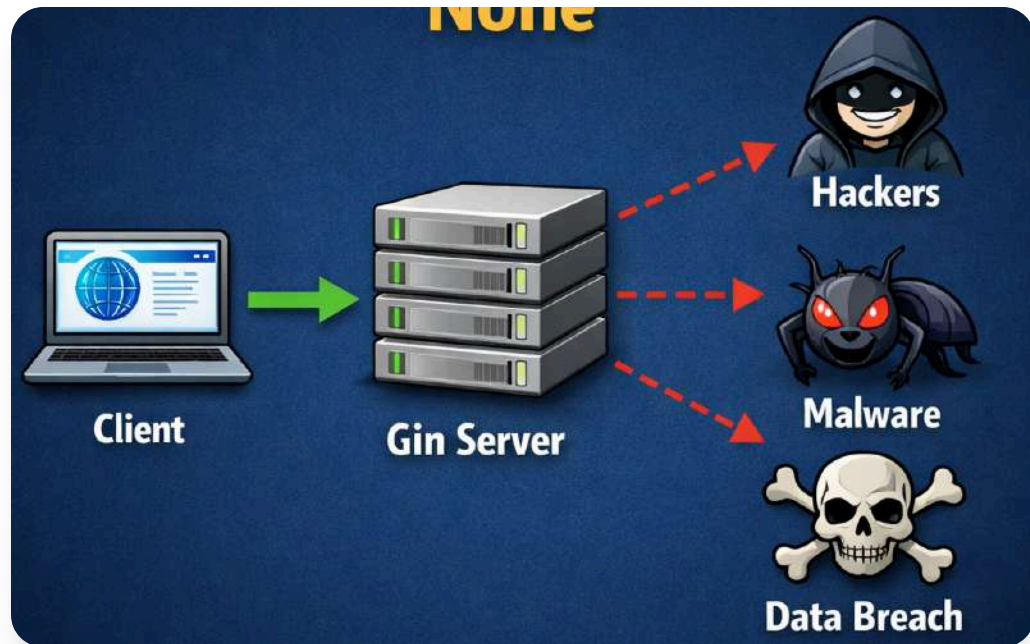
03 CORS dan satu perintah jalan

Tambahkan konfigurasi *CORS* untuk frontend lokal, lalu pastikan seluruh sistem dapat dijalankan dengan satu perintah Compose.

02 Compose untuk app dan database

Siapkan *docker-compose.yml* agar service aplikasi dan PostgreSQL berjalan bersama dalam satu lingkungan.

```
ports:
- "8080:8080"
environment:
- "PORT=8080"
depends_on:
- "db"
restart: always
db:
image: "postgres:13"
environment:
- "POSTGRES_DB=mydatabase"
- "POSTGRES_USER=myuser"
- "POSTGRES_PASSWORD=mypassword"
volumes: "pg_data:/var/lib/pgsql/data"
```



Ringkasan

Backend Gin perlu aman, terkendali, dan mudah dideploy dengan fondasi security, CORS, limiter, dan Docker.